KV-Cache Compression in Llama-3.2

Compact technical report

Date: 2026-08-11 (revised 2026-08-12)
Model: `unsloth/Llama-3.2-1B-Instruct`
Hardware: NVIDIA RTX 4090
Scope: Low-rank, tensor, quantized, and behavior-aware compression of the inference KV cache

---

Executive summary

This project asked whether an LLM's KV cache can be compressed effectively with matrix or tensor low-rank decomposition. In the tested regime, the answer was **no**: keys had moderate low-rank structure, but values did not, and low Frobenius reconstruction error did not reliably preserve attention or model output.

The experiments moved from structural diagnostics to next-token behavior, multi-token reasoning, physical byte accounting, and strict causal attention measurements. The main findings were:

- At 95% retained energy, the mean required rank was 30.06/64 for keys and 49.44/64 for values.
- Attractive SVD ranks caused large output-distribution changes; jointly compressing K and V at rank 16 produced next-token KL 3.13.
- Reconstructing dense K/V from factors reduced stored bytes but increased latency and energy.
- Tucker and Tensor Train did not improve the tested quality-memory frontier.
- Attention-influence residuals were far better than magnitude- or energy-based residuals, but neither low rank nor INT2 plus repair beat uniform INT4.
- Strict causal diagnostics showed that K repair improved routing, but aggressive V2 quantization dominated the remaining attention-output error.
- A final mixed V2/V4 codec used exactly the same aged-cache bytes as K4/V4 and remained more complex with worse attention-output fidelity.

The practical endpoint is therefore

$$\boxed{\text{uniform K4/V4 quantization.}}$$

No specialized low-rank or mixed-precision kernel is justified by the measured quality-byte frontier.

---

1. Problem and notation

For one attention head with n cached tokens,

$$K \in \mathbb{R}^{n \times d}, \qquad V \in \mathbb{R}^{n \times d_v}, \qquad q \in \mathbb{R}^{d}.$$

Attention computes

$$z = \frac{Kq}{\sqrt{d}}, \qquad p = \mathrm{softmax}(z), \qquad y = V^\top p = \sum_{i=1}^{n} p_i v_i.$$

Keys determine where attention is routed; values determine what content is returned. During autoregressive generation, past K and V are cached so they are not recomputed at every step.

The tested model has 16 layers, 32 query heads, 8 KV heads, and head dimension $d = 64$. Four query heads share each KV head. At sequence length S, each layer stores K and V with shape `[1, 8, S, 64]`. In BF16, this costs

$$2\ (\text{K,V}) \times 8 \times 64 \times 2\ \text{bytes} = 2{,}048\ \text{bytes/token/layer},$$

or 32 KiB/token across 16 layers. The resulting cache is 64 MiB at 2K context, 256 MiB at 8K, and approximately 4 GiB at 131K.

---

2. Why reconstruction error is the wrong objective

2.1 Low-rank candidates

For $X \in \mathbb{R}^{S\times d}$, truncated SVD gives

$$X \approx U_r \Sigma_r V_r^\top = AB^\top$$

and minimizes $\|X - \widehat{X}\|_F$ over rank-r matrices. It stores approximately $r(S+d)$ rather than Sd numbers. Flattened SVD applies the same idea to $X_{(S)} \in \mathbb{R}^{S\times(HD)}$, sharing a token basis across heads.

For $X \in \mathbb{R}^{H\times S\times D}$, Tucker uses

$$X \approx G \times_1 U_H \times_2 U_S \times_3 U_D$$

with storage $Hr_H + Sr_S + Dr_D + r_H r_S r_D$. Tucker helps only if the realized tensor has separable low-rank structure across several modes; reshaping a matrix into a tensor does not create such structure.

2.2 Exact propagation of compression error

Let

$$\widehat{K} = K + E_K, \qquad \widehat{V} = V + E_V,$$

and define

$$\hat{p} = \mathrm{softmax}\left(z + \frac{E_K q}{\sqrt{d}}\right).$$

The attention-output error decomposes exactly as

$$\hat{y} - y = V^\top(\hat{p} - p) + E_V^\top \hat{p}.$$

The first term is **K-routing error**: key error changes the attention distribution before values are combined. The second is **V-content error**: value error is averaged using the compressed attention weights. For small errors, with

$$J_p = \mathrm{diag}(p) - pp^\top,$$

the first-order perturbation is

$$\delta y \approx V^\top J_p \frac{E_K q}{\sqrt{d}} + E_V^\top p.$$

This is the central mismatch: Frobenius loss weights every cache entry and direction equally, whereas attention weights key error by realized queries, score margins, softmax geometry, and value differences, and weights value error by attention probability.

2.3 Key sensitivity

For token i, the score error is

$$\delta z_i = \frac{q^\top e_{k_i}}{\sqrt{d}}.$$

Only the query-aligned component matters. If $q^\top e_{k_i} = 0$, even a large key error produces no score error for that query. Across queries with covariance $\Sigma_q = \mathbb{E}[qq^\top]$, the relevant quadratic distortion is

$$\mathbb{E}_q\left[\left\|\frac{E_K q}{\sqrt{d}}\right\|_2^2\right] = \frac{1}{d}\operatorname{tr}(E_K \Sigma_q E_K^\top),$$

not merely $\|E_K\|_F^2$.

Conversely, let tokens i and j have score margin $m = z_i - z_j > 0$. Their ordering reverses if

$$\delta z_j - \delta z_i > m.$$

An arbitrarily small perturbation can therefore change the top-attended token when scores are nearly tied. Softmax is also invariant to common shifts, so $\delta z = c\mathbf{1}$ changes no probability. Relative score errors, not raw key error, determine routing.

2.4 Value sensitivity

If K is exact, then

$$\hat{y} - y = E_V^\top p = \sum_i p_i e_{v_i}, \qquad \frac{\partial y}{\partial v_i} = p_i I,$$

and

$$\|\hat{y} - y\|_2 \le \sum_i p_i \|e_{v_i}\|_2.$$

Thus equal-sized value errors generally matter more at highly attended tokens. Low-probability errors can still accumulate, but uniform reconstruction loss does not represent their functional weighting.

2.5 Storage is not execution

Although $K \approx AB^\top$ stores $r(n+d)$ numbers, explicitly materializing $AB^\top$ restores an $n \times d$ tensor and adds $O(ndr)$ reconstruction work. A useful implementation must operate on factors directly:

$$Kq \approx A(B^\top q), \qquad V^\top p \approx D(C^\top p) \quad \text{for } V \approx CD^\top.$$

These paths cost $O(r(n+d))$ arithmetic but require factor-aware kernels. A compact stored representation that is decoded to dense K/V before stock attention is a quality prototype, not an end-to-end memory or speed result.

---

3. Experimental progression

Each stage tested a stronger claim than the preceding proxy.

Stage	Question	Decisive outcome
1	Are K or V low rank?	K was moderately low rank; V required much higher rank.
2	Does reconstruction preserve the next token?	Attractive ranks produced large output KL and changed top tokens.
3	Do stored factors improve the system?	Dense reconstruction increased latency and energy.
4	Are K and V equally sensitive?	K had lower reconstruction error but greater routing sensitivity.
5	Do tensor models or pre-RoPE fitting help?	Tucker/TT did not improve the frontier; pre-RoPE K was more compressible but did not solve execution.
6	Can an exact hot window and sparse residual repair low rank?	Attention influence greatly outperformed geometric selectors.
7	Does repair survive free generation?	Influence repair recovered much of GSM8K quality; low rank alone failed.
8	What wins at matched packed bytes?	Uniform INT4 dominated low-rank candidates.
9	Can influence-repaired INT2 beat INT4?	No; a future-query oracle nearly could, exposing a causal prediction bottleneck.
10	Is remaining error mainly in K or V?	K repair worked, but V2 content error dominated.
11	Can value-aware mixed V2/V4 win at the INT4 budget?	It approached but did not dominate K4/V4 and was more complex.

Early stages used single prompts and next-token metrics to select directions. Later stages used three held-out WikiText windows at both 2K and 8K context, 256 teacher-forced evaluation positions per window, complete representation-byte accounting, and strict causal block aging. GSM8K tests used 50 or 100 examples and are treated as small-sample diagnostics, not equivalence tests.

---

4. Results

4.1 Structure, behavior, and systems cost

Per-head SVD on a 441-token prompt across all 16 layers and 8 KV heads gave:

Energy retained	Mean K rank	Mean V rank	Maximum rank
90%	20.75	40.25	64
95%	30.06	49.44	64
99%	49.19	60.31	64

Rank	K relative error	V relative error	Analytic factor compression
4	0.560	0.811	13.97×
8	0.483	0.724	6.99×
16	0.369	0.595	3.49×

Rank	K relative error	V relative error	Analytic factor compression
32	0.205	0.401	1.75×

K was structurally more compressible, but structural fit did not predict behavior. Joint K/V compression produced:

Rank	Next-token KL	Top token preserved?
4	4.82	No
8	4.85	No
16	3.13	No
32	0.86	Yes
64	0.00039	Yes

At 2K context, K-only rank 16 had final-output KL 3.275, whereas V-only rank 16 had KL 0.830; at 8K, the corresponding values were 4.81 and 0.127. K was easier to reconstruct yet more dangerous to perturb because it controlled routing.

The reconstruction-based prototype also failed as a system. At 4K context:

Variant	Stored prefill KV	TTFT	TPOT	Energy/token
Dense	128 MiB	527.5 ms	21.04 ms	6.34 J
Rank-16 factors	32.5 MiB	996.6 ms	24.29 ms	7.29 J
Rank-32 factors	65.0 MiB	970.9 ms	22.04 ms	6.90 J

Rank 16 reduced stored KV by 3.94×, but increased time to first token by 89%, time per output token by 15.5%, and energy/token by 15%.

4.2 Tensor structure and RoPE

At 2K context, per-head rank-32 SVD achieved 1.94× compression with KL 0.698. Tucker ranks `[8, 256, 32]` achieved 1.77× with KL 4.650, and TT ranks `[8, 32]` achieved 1.99× with KL 4.741.

On `[8, 256, 64]` tensors at a rank-16-comparable budget, flattened-SVD/Tucker errors were 0.20015/0.19994 for K and 0.40517/0.40508 for V. Tucker selected ranks `[8, 53, 64]`, retaining full head and feature ranks and compressing only the token mode. It therefore reduced to nearly the same approximation class as token-flattened SVD.

RoPE also altered apparent rank. Mean rank for 95% key energy was 13.88 before RoPE and 26.14 after RoPE. At rank 16, pre-RoPE reconstruction error was 0.227 versus 0.302 for direct post-RoPE SVD. This reveals latent structure but does not remove the need to apply position-dependent rotations efficiently during attention.

4.3 Behavior-aware sparse repair

Stage 6 used a 64-token exact hot window, fixed calibration bases for old blocks, and sparse exact residuals. At rank 16 and 5% residual density:

Selector	Mean KL over three windows
No residual	1.177
Random	1.113
Magnitude	1.020
Energy	1.003
Attention influence	**0.0407**

Influence used observed attention mass and query sensitivity. Its advantage over same-size geometric selectors showed that approximation error was highly nonuniform in functional importance.

The same pattern survived free generation on 100 GSM8K examples:

Cache method	Old bytes/token/layer	Compression	Accuracy
Full	2,048	1×	39%
Fixed latent rank 16	512	4×	2%
Rank 16 + influence 2%	593.9	3.45×	30%
Rank 16 + influence 5%	716.8	2.86×	38%
Rank 16 + energy 5%	716.8	2.86×	3%
INT2 old cache	384	5.33×	17%

The dense and exact-copy controls both scored 39/100 and generated identical outputs. The result supports influence selection, not low rank as a complete codec.

4.4 Matched-byte frontier

With a 64-token hot window, 256 evaluation tokens, and physically packed old-cache representations, Stage 8 found:

Method	Old bytes/token/layer	KL at 2K	KL at 8K
INT2 K/V	384	0.1363	0.1523
Latent-16 K/V	512	1.1119	1.1932
Latent-16 K + INT4 V	576	0.8231	0.8801
INT4 K/V	**640**	**0.00450**	**0.00482**
Latent-16 K + INT4 V + 5% K residual	678.4	0.0745	0.1260
Latent-32 K + INT4 V + 5% K residual	934.4	0.0337	0.0577

INT4 preserved all directions with small coordinate noise; low rank removed complete directions. In this cache, retaining directions was more important than high-precision coordinates.

Replacing the low-rank bulk with INT2 improved the hybrid but still did not beat INT4:

Method	Bytes/token/layer	KL at 2K	KL at 8K
INT4	640	**0.00450**	**0.00475**
INT2	384	0.13627	0.15262
INT2 + influence 2%	465.9	0.02557	0.04099
INT2 + influence 5%	588.8	0.02038	0.02899
INT2 + exact rows 5%	588.8	0.02542	0.03885
INT2 + energy 5%	588.8	0.07116	0.07719

At 588.8 bytes, a true future-query oracle achieved mean KL 0.00449, nearly matching INT4; causal past-attention and query-magnitude selectors achieved 0.02420 and 0.07020. The representation had sufficient repair capacity, but predicting future importance from past information was the bottleneck. Future-query selection would be leakage.

4.5 Strict causal diagnosis

Stage 10 compressed 256-token blocks exactly when they left a 64-token hot window, retained the first four tokens exactly, and evaluated three held-out windows at 2K and 8K over 256 positions with per-layer/head diagnostics. It compared uniform K4/V4 with K4/V2 plus a causal K-only influence residual.

Context	Method	Complete bytes	Compression	Final KL	Attention-output error
2K	K4/V4	816.0	2.51×	0.00155	0.01332
2K	K4/V2 + K repair 2%	739.8	2.77×	0.00558	0.04144
2K	K4/V2 + K repair 5%	793.6	2.58×	0.00535	0.04107
8K	K4/V4	684.0	2.99×	0.00212	0.01470
8K	K4/V2 + K repair 2%	599.7	3.42×	0.00671	0.04618
8K	K4/V2 + K repair 5%	659.2	3.11×	0.00694	0.04578

For the 5% candidate, the mechanistic decomposition was:

Context	Method	K-routing error	V-content error	Total error
2K	K4/V4	0.01032	0.00744	0.01332
2K	K4/V2 + K repair	0.00650	0.03982	0.04107
8K	K4/V4	0.01134	0.00798	0.01470
8K	K4/V2 + K repair	0.00793	0.04398	0.04578

K repair reduced attention KL, top-attention disagreement, and routing error below K4/V4. The candidate nevertheless failed because V2 content error was about 5.4–5.5× the K4/V4 value error, making total attention-output error about 3× worse.

4.6 Equal-budget mixed V2/V4

The final codec retained K4 plus a 2% K residual and assigned each 32-value V group either two or four bits. If V_2 and V_4 are the candidate reconstructions, the reducible value error was

$$g = \|V - V_2\|_2 - \|V - V_4\|_2,$$

and the causal selection score was

$$s = (\text{past attention mass})\, g.$$

After payloads, FP16 scales/minima, a one-bit width mask, K residual values, and indices, the exact 640-byte aged-cache budget allowed 66.46% of V groups to use four bits.

All methods below used 640 aged bytes/token/layer:

Method	Final KL (2K/8K)	Attention KL (2K/8K)	Output error (2K/8K)
K4/V4	0.00155 / 0.00212	0.00139 / 0.00250	0.01332 / 0.01470
Mixed, random	0.00259 / 0.00366	0.00104 / 0.00197	0.02379 / 0.02570
Mixed, error only	0.00151 / 0.00193	0.00103 / 0.00198	0.01711 / 0.01969
Mixed, attention aware	0.00152 / 0.00213	0.00103 / 0.00198	0.01366 / 0.01673

Attention-aware promotion was the best mixed selector on attention-output error. It was still 2.6% worse than K4/V4 at 2K and 13.8% worse at 8K, at identical aged bytes, while requiring two packed streams, a width mask, sparse K corrections, and a causal selector. It therefore failed the predefined gate.

5. Interpretation and decision

Ordinary low-rank KV compression failed for five connected reasons:

1. **V was not sufficiently low rank.** At 95% energy it required mean rank 49.44/64.
2. **Frobenius loss weighted the wrong errors.** Attention depends on query alignment, score margins, softmax probabilities, and value content.
3. **K was structurally compressible but behaviorally sensitive.** Low average reconstruction error did not protect rare query-aligned routing directions.
4. **The tested tensor axes added little separability.** Tucker retained full head and feature ranks and mostly compressed the token mode.
5. **Dense reconstruction erased systems gains.** Smaller stored factors did not imply lower peak memory, bandwidth, latency, or energy.

Influence selection remains a useful result: past attention and query information identified important errors far better than magnitude or energy, and sparse repair measurably improved K routing. It did not produce a superior complete codec because causal prediction could not match the future-query oracle and aggressive V compression remained costly.

The final choice follows strict Pareto reasoning. Relative to uniform K4/V4, the strongest mixed codec used the same aged-cache bytes, had worse attention-output fidelity, and required a more complicated representation and kernel. Therefore:

> Reject ordinary low-rank KV decomposition in this regime.
> Use uniform K4/V4 as the current practical endpoint.

---

6. Limits of the evidence

- **Model coverage:** Most experiments used one 1B Llama model; results need not transfer to larger models, different head dimensions, or other architectures.
- **Domain coverage:** Later stages used three WikiText windows per context, not broad code, dialogue, retrieval, mathematical, or multilingual corpora.
- **Dependence:** Positions within a continuation are correlated; uncertainty should be estimated by document/window resampling, not by treating tokens as independent.
- **Task power:** GSM8K samples of 50–100 have wide paired confidence intervals. A 44/100 versus 45/100 result does not establish equivalence.
- **Harness drift:** Dense GSM8K accuracy changed from 39% to 45% across runs, so cross-run comparisons require pinned model, prompt, tokenizer, generation settings, operation order, and environment.
- **Systems scope:** Stages 8–11 counted physical representation bytes but decoded to dense tensors for stock attention. They establish quality and representation size, not actual peak-memory reduction or speedup.

---

7. Recommended next work

7.1 Engineering: make K4/V4 genuinely packed

The experiments established that K4/V4 had the best representation-level tradeoff. The next engineering question is whether it also saves memory and time when attention actually runs. The old cache must remain four-bit on the GPU; writing a dense BF16 copy before every attention call would restore most of the memory traffic the codec is meant to remove.

For a group G of cache values, a simple unsigned affine four-bit codec is

$$s_G = \frac{x_{\max,G} - x_{\min,G}}{15}, \qquad c_j = \operatorname{clip}_{[0,15]}\left(\operatorname{round}\frac{x_j - x_{\min,G}}{s_G}\right), \qquad \hat{x}_j = x_{\min,G} + s_G c_j.$$

Each code c_j uses four bits, so two codes fit in one byte. The scale and minimum are stored once per group. Other symmetric or asymmetric INT4 layouts are possible, but the candidate and baseline must use the same grouping and include all metadata in the byte count.

The attention kernel should read a small tile of packed keys and values, unpack and dequantize it in registers or shared memory, and immediately use it in

$$z_i = \frac{q^\top \hat{k}_i}{\sqrt{d}}, \qquad \hat{y} = \sum_i \operatorname{softmax}(z)_i \hat{v}_i.$$

It should never write the full $\widehat{K}$ or $\widehat{V}$ to global GPU memory. A numerically stable tiled softmax can maintain a running maximum m, normalization ℓ, and unnormalized output o. For a new tile T,

$$\begin{aligned} m' &= \max\left(m, \max_{j\in T} z_j\right), \\ \ell' &= e^{m-m'}\ell + \sum_{j\in T} e^{z_j - m'}, \qquad \hat{y} = \frac{o}{\ell}. \\ o' &= e^{m-m'}o + \sum_{j\in T} e^{z_j - m'}\hat{v}_j, \end{aligned}$$

This fuses dequantization with attention: a cache tile is decoded only while it is being consumed.

A small exact recent window of W tokens may remain in BF16 because recent tokens often receive high attention and have not yet amortized their encoding cost. For one layer with H KV heads and $d = d_v$, an exact K/V token costs

$$b_{16} = 2\ (\text{K,V}) \times H \times d \times 2\ \text{bytes} = 4Hd,$$

which is 2,048 bytes for $H = 8, d = 64$. The four-bit K/V payload costs $b_{4,\text{payload}} = Hd = 512$ bytes/token before metadata. At context n, complete resident cache memory is therefore

$$B_{\text{resident}} = (n - W)(b_{4,\text{payload}} + b_{4,\text{metadata}}) + W b_{16} + B_{\text{padding}},$$

and the end-to-end peak is

$$B_{\text{peak}} = \max_t \left(B_{\text{model}} + B_{\text{resident}}(t) + B_{\text{workspace}}(t) + B_{\text{allocator}}(t)\right).$$

This distinction matters: "512 payload bytes" is not a system memory result if scales, padding, a hot window, or a large temporary buffer are omitted.

The implementation should be compared with dense BF16 and an established packed INT4/KIVI path using identical prompts, batch sizes, and generation lengths. Report:

- **encoding cost:** time and energy to quantize each block when it leaves the hot window;
- **bytes read per generated token:** measured with a GPU profiler, not inferred only from tensor shapes;
- **time to first token (TTFT):** request start to the first generated token, including prefill;
- **time per output token (TPOT):** $(t_{\text{last}} - t_{\text{first}})/(N_{\text{generated}} - 1)$;
- **energy per token:** $\int P(t)\,dt/N_{\text{generated}}$, with the same idle-power convention for every method;
- **peak allocated and reserved memory:** measured over the entire request.

A useful engineering result must show both a quality pass and a real reduction in peak memory or memory traffic without an unacceptable TTFT/TPOT regression.

7.2 Validation: show that quality is not meaningfully worse

"Pinned process" means that dense and compressed runs share one model instance and exactly the same weight revision, tokenizer, prompts, seeds, batch order, decoding settings, software versions, and fixed kernel implementations. This prevents the 39%-to-45% baseline drift observed across earlier GSM8K runs from being mistaken for a codec effect.

Use the full GSM8K set for multi-step reasoning, long-context retrieval to test whether distant information is still found, instruction-following tests, and several natural-text domains. Repeat at increasing valid context lengths on Llama-3.2-1B, Llama-3.2-3B, and at least one different architecture. This separates a general compression effect from an accident of one model or dataset.

Every comparison should be paired: item j is evaluated once with the baseline and once with the compressed cache. For accuracy, define

$$d_j = \mathbf{1}\{\text{compressed correct on } j\} - \mathbf{1}\{\text{baseline correct on } j\}, \qquad \overline{d} = \frac{1}{N}\sum_{j=1}^{N} d_j.$$

Choose an acceptable degradation $\delta_{\text{acc}} > 0$ before seeing the results. Non-inferiority is supported only if the lower confidence bound satisfies

$$\text{LCB}_{1-\alpha}(\overline{d}) > -\delta_{\text{acc}}.$$

For an error or loss, where smaller is better, use paired differences $d_j = L_{\text{compressed},j} - L_{\text{baseline},j}$ and require

$$\text{UCB}_{1-\alpha}(\overline{d}) < \delta_{\text{loss}}.$$

Thus "the difference was not statistically significant" is not enough; the interval must exclude a degradation larger than the predeclared tolerance.

Token errors within a document are correlated. Bootstrap whole documents or context windows: sample windows with replacement, keep all positions within each sampled window, recompute the paired difference, and use the bootstrap quantiles as the confidence interval. Also report the median and 95th/99th percentiles of attention KL and relative output error. A small mean can hide rare large failures that dominate autoregressive generation.

Finally, report complete physical bytes:

$$B_{\text{complete}} = B_{\text{payload}} + B_{\text{scales/minima}} + B_{\text{masks}} + B_{\text{indices}} + B_{\text{padding}} + B_{\text{hot}} + B_{\text{workspace}}.$$

Quality comparisons should be made at matched B_{complete}, not matched nominal bit width.

7.3 Research: change the objective, information, or execution model

Adding more tensor modes while still minimizing $\|X - \widehat{X}\|_F^2$ does not address the demonstrated mismatch. A new method should optimize a quantity closer to how the cache is used. With held-out queries, possible objectives are

$$\mathcal{L}_{\text{score}} = \mathbb{E}_q \frac{\|(K - \widehat{K})q\|_2^2}{d},$$

$$\mathcal{L}_{\text{attn}} = \mathbb{E}_q D_{\text{KL}}\left(\text{softmax}\,\frac{Kq}{\sqrt{d}}\,\middle\|\,\text{softmax}\,\frac{\widehat{K}q}{\sqrt{d}}\right),$$

or the joint attention-output objective

$$\mathcal{L}_{\text{out}} = \mathbb{E}_q \frac{\|V^\top p - \widehat{V}^\top \hat{p}\|_2^2}{\|V^\top p\|_2^2 + \varepsilon}.$$

Queries used to choose ranks, bases, residuals, or bit widths must be separated from the queries used for evaluation.

Another direction is causal prediction of future importance. When a block is compressed at time t, the selector may use only history $\mathcal{H}_t$, not future queries. For group g, learn

$$I_g(t) = \mathbb{E}[\text{future attention loss caused by } g \mid \mathcal{H}_t],$$

then choose protected groups S by the budgeted problem

$$\max_S \sum_{g\in S} I_g(t) \quad \text{subject to} \quad \sum_{g\in S} b_g \le B_{\text{repair}}.$$

The predictor may be trained on separate sequences where future queries are known, but at deployment it must use only $\mathcal{H}_t$. Other credible changes include training the model to produce a compressible cache, changing the attention architecture, or computing attention directly from factors rather than reconstructing dense tensors.

A per-query certificate could make an adaptive codec safer. Let

$$\eta = \frac{\|E_K q\|_\infty}{\sqrt{d}}.$$

If the compressed top-score margin $\hat{\gamma}$ satisfies $\hat{\gamma} > 2\eta$, the identity of the exact top-scoring key is guaranteed to be unchanged. Quantization metadata can give a conservative bound without storing dense K: if coordinate j of key i has error at most Δ_{ij}, then

$$\eta \le \max_i \frac{1}{\sqrt{d}} \sum_j |q_j| \Delta_{ij}.$$

The full output error also obeys

$$\|\hat{y} - y\|_2 \le \max_i \|v_i\|_2 \, \|\hat{p} - p\|_1 + \sum_i \hat{p}_i \|e_{v_i}\|_2.$$

With $\|E_K q/\sqrt{d}\|_\infty \le \eta$, one conservative softmax bound is

$$\|\hat{p} - p\|_1 \le \min\{2, e^{2\eta} - 1\}.$$

If a certificate exceeds a chosen tolerance, the system could fetch an exact residual or refine the affected block from two to four bits. Such refinement must be causal and its extra bytes and latency must be counted.

Before investing in a specialized kernel, test a correctness prototype at matched complete bytes. A candidate advances only if it is uncertainty-aware Pareto competitive with K4/V4: no more bytes with reliably lower error, or fewer bytes while satisfying predeclared non-inferiority bounds. Kernel work then asks whether the representation-level gain survives real execution.

Appendix A. Metrics

Metric	Definition and interpretation
Relative Frobenius error	$\|X - \widehat{X}\|_F / \|X\|_F$; geometric reconstruction error, not behavior.
Final-logit KL	$D_{\mathrm{KL}}(p_{\mathrm{dense}} \| p_{\mathrm{compressed}})$; disagreement between next-token distributions.
Perplexity	exp(mean negative log likelihood); predictive fit to target tokens.
Attention KL	KL between dense and compressed attention distributions.
Top-attention disagreement	Fraction of queries whose most-attended token changes.
Attention-output error	$\|\hat{y} - y\|_2 / \|y\|_2$; error passed to the rest of the layer.
Complete bytes/token/layer	Payload, scales, masks, indices, exact regions, padding, and incomplete blocks divided by tokens and layers.

Appendix B. Reproducibility artifacts

- Stages 1–8: Stage 1 results through Stage 8 results
- Stages 9–11: Stage 9, Stage 10, and Stage 11
- GSM8K: Stage 7 hybrid, Stage 7b quantization, and Stage 9b repair
- Tensor anatomy demo
- Earlier consolidated narrative